Predicting Diabetes Disease Progression

OLS, Ridge and Lasso Regression with Feature Engineering

Andhur Rahul Nair
EO ID: x3arn123

ePGD in AI & DS, Semester 2 | Machine Learning | Mid-Semester Project

---

Summary

In this project I looked at how well regression models can predict diabetes disease progression one year after a patient's baseline measurements. I used the load_diabetes dataset from scikit-learn, which has 442 patients and 10 baseline features. I didn't want to just chase the lowest error. I wanted to understand how feature engineering, model complexity and regularization change what OLS, Ridge and Lasso regression can do.

What I found. BMI and s5 (log serum triglycerides) have the strongest link with progression, followed by blood pressure, s4, HDL (s3, the only negative one) and blood sugar (s6). A linear model explains about half of the variation, with a cross-validated R^2 of 0.48. On the original 10 features, regularization doesn't make predictions more accurate, but it does steady the coefficients of the strongly correlated lipid features. Nonlinear models (Random Forest, Gradient Boosting, SVR) didn't beat the linear model, and adding squares, cubes and interaction terms didn't help either. OLS overfits badly as the polynomial degree goes up, Ridge degrades gently, and Lasso holds up best. In the end, I'd pick Ridge regression on the original features. On the held-out test set it reaches an R^2 of 0.454, with an RMSE of 53.8 and an MAE of 42.8.

1. Project Objective

This is a regression problem because the target is a continuous number. The goal is to learn a function f: R^10 → R so that ŷ = f(x) is a good estimate of the progression score y from a patient's baseline characteristics x. The question I kept coming back to throughout the project was:

How do feature engineering, model complexity and regularization affect the ability of OLS, Ridge and Lasso regression to predict diabetes disease progression?

2. Data and Exploratory Analysis

2.1 Dataset

The dataset has 442 individuals and 10 baseline features covering demographic, physiological and biochemical measurements. I checked for problems first: there are no missing values and no duplicate rows. scikit-learn already mean-centres and scales the features (each has a standard deviation of

about 0.048), so the raw coefficients are on a comparable scale. The target has a mean of 152.1 and a standard deviation of 77.1, and it ranges from 25 to 346.

Feature	Description
age	Age of the individual
sex	Sex of the individual
bmi	Body Mass Index
bp	Average blood pressure
s1	Total serum cholesterol
s2	Low-density lipoproteins (LDL)
s3	High-density lipoproteins (HDL)
s4	Total cholesterol / HDL ratio
s5	Log-transformed serum triglycerides level
s6	Blood sugar level

Table 1. Features in the diabetes dataset.

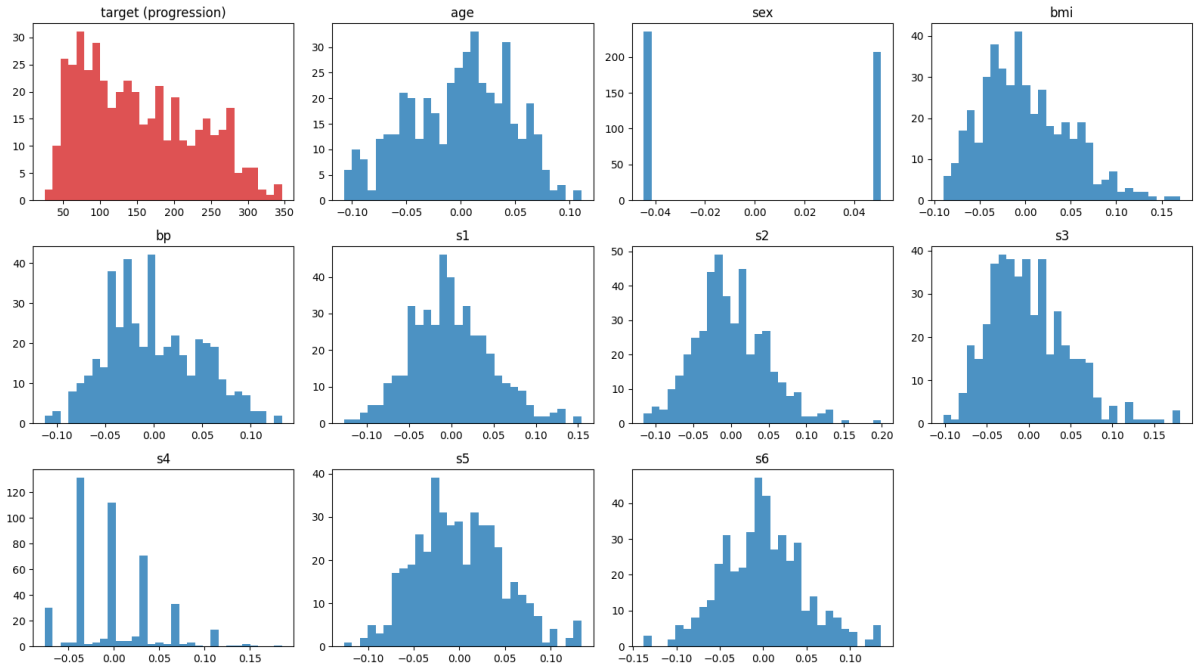


Figure 1. Distributions of the target (top left) and the ten features.

2.2 Correlations and multicollinearity

Figure 2 shows the correlation matrix for the features and the target. A few of the serum measurements are strongly correlated with each other, most clearly s1 and s2 (about 0.9). The variance inflation factors (VIF) in Table 2 back this up. s1 (59.2), s2 (39.2), s3 (15.4), s5 (10.1) and s4 (8.9) are strongly collinear, while every other feature is below 1.6. This turned out to matter later, because collinearity makes individual OLS coefficients unstable.

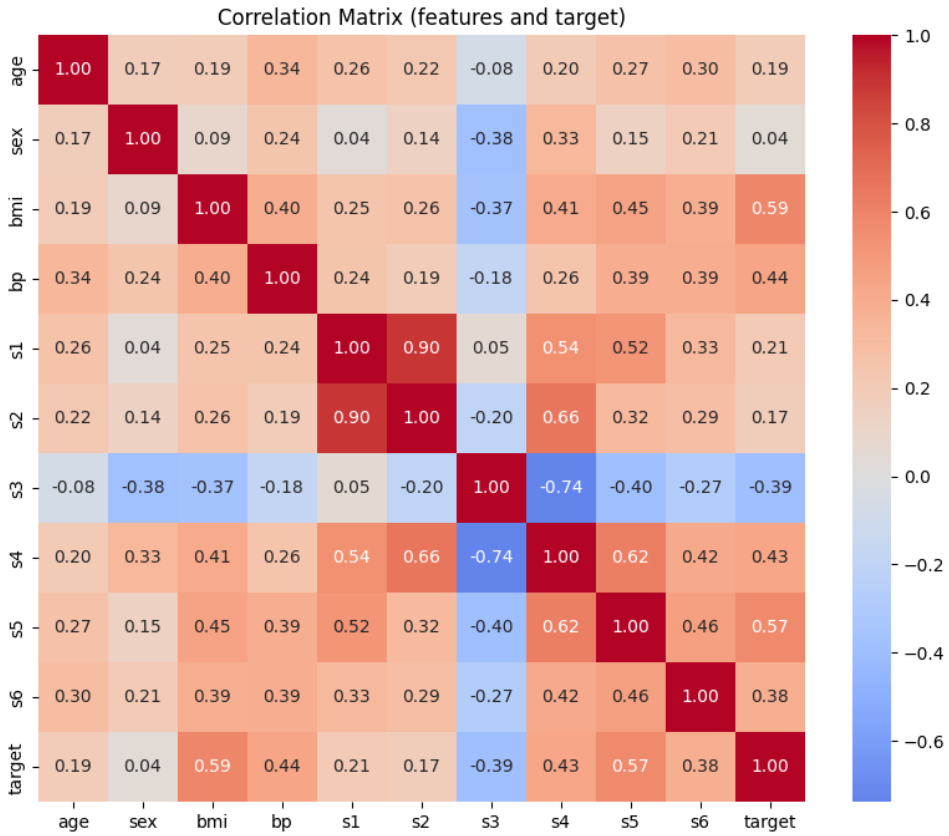


Figure 2. Correlation matrix of the features and the target.

Feature	VIF	Feature	VIF
s1	59.20	bmi	1.51
s2	39.19	s6	1.48
s3	15.40	bp	1.46
s5	10.08	sex	1.28
s4	8.89	age	1.22

Table 2. Variance inflation factors (VIF above 5-10 means strong multicollinearity).

3. Methodology

Training and testing. I split the data once into a training set (80%, 353 rows) and a test set (20%, 89 rows). Everything that involved choosing or tuning a model, including the Ridge and Lasso penalty strengths, used only the training data and 5-fold cross-validation. I used the same shuffled folds for every model so the comparisons are fair. I only used the test set at the end, to report how the models do on unseen data, and never to choose between them. I used one random seed (42) throughout so the results can be reproduced.

Models. My baseline is ordinary least squares (OLS), which has no penalty. Ridge adds an $\ell 2$ penalty that shrinks the coefficients and keeps them steadier when predictors are correlated. Lasso adds an $\ell 1$ penalty that can set coefficients exactly to zero, so it regularizes and selects features at the same time. For comparison I also fit three nonlinear models: Random Forest, Gradient Boosting and an RBF-kernel SVR. I standardise the features inside each pipeline so the penalties treat them all equally.

Feature engineering. For the engineered feature set I added the square and cube of every feature plus all 45 pairwise interactions, which gives 75 features (10 + 10 + 10 + 45). To see what happens as complexity grows, I also used full polynomial expansions of degree 1, 2 and 3 (10, 65 and 285 features).

Metrics. I report R^2, RMSE and MAE. Cross-validated R^2 is my main criterion for choosing a model, and the gap between training R^2 and CV R^2 tells me how much a model overfits.

4. Results: Research Questions

Q1. Which baseline characteristics have the strongest association with progression?

BMI (+0.59) and s5 (+0.57) are clearly the strongest. Blood pressure (+0.44), s4 (+0.43), s3 (−0.39) and s6 (+0.38) come next with moderate associations. s3 (HDL) is the only negatively correlated feature, so higher HDL goes with less progression, while every other feature goes the other way. One caveat is that these are simple one-feature-at-a-time Pearson correlations. They show association, not causation.

Feature	bmi	s5	bp	s4	s6	s1	age	s2	sex	s3
Correlation with target	0.586	0.566	0.441	0.430	0.382	0.212	0.188	0.174	0.043	−0.395

Table 3. Pearson correlation of each feature with the target.

Q2. How well can a linear model explain the variation in the target?

On the training data, OLS gets a 5-fold CV R^2 of 0.480 (± 0.041), with a CV RMSE of about 55.4 and a CV MAE of about 45.0. So a linear model explains roughly half of the variation in disease progression. The other half could be noise, predictors we don't have, or non-linear effects, and R^2 on its own can't tell me which. The final model's test R^2 (0.454) is close to the CV estimate, so the CV number was a fair guide.

Q3. Does regularization improve generalization when predictors are correlated?

Not for accuracy. OLS, Ridge and Lasso all land at a CV R^2 of about 0.48 and a CV RMSE of about 55.4. The differences are tiny next to the fold-to-fold spread of about 0.04. With only 10 predictors and 353 training rows, OLS doesn't have much variance for regularization to remove.

Model	CV R^2	R^2 std	CV RMSE	CV MAE
OLS	0.48	0.041	55.395	45.048
Ridge	0.48	0.041	55.419	45.119
Lasso	0.48	0.040	55.394	45.180

Table 4. OLS vs Ridge vs Lasso, 5-fold CV on the training data (original features).

Where regularization does help is stability, as Table 5 shows. The shaky OLS coefficients of the collinear features get shrunk. s1 goes from −44.4 (OLS) to −34.7 (Ridge) to −8.2 (Lasso), s2 goes from 24.6 to 17.1 to 0.0, and s5 goes from 35.2 to 31.4 to 22.6. Lasso also flips s3 from +7.7 to −9.0. I read that as Lasso picking one feature from a group of correlated ones more or less arbitrarily, not as a real negative effect. The less collinear features (bmi, bp, sex, s6) barely change.

Feature	OLS	Ridge	Lasso
age	1.8	1.8	0.7
sex	−11.5	−11.4	−9.3
bmi	25.6	25.7	26.2
bp	16.8	16.7	15.7
s1	−44.4	−34.7	−8.2
s2	24.6	17.1	0.0
s3	7.7	3.4	−9.0
s4	13.1	11.8	3.4
s5	35.2	31.4	22.6
s6	2.4	2.5	2.1

Table 5. Coefficients on standardised features, fitted on the training data (selected alpha = 1.0 for both Ridge and Lasso).

Q4. Do nonlinear models capture relationships that a linear model cannot?

I didn't find any evidence of that. SVR with an RBF kernel (R^2 0.475) is tied with OLS (0.480), while Random Forest (0.432) and Gradient Boosting (0.423) come out about 0.05 to 0.06 lower. The polynomial models in Section 5 point the same way, since adding non-linear terms never beat the plain linear fit. My reading is that any non-linear structure is small compared with the noise. The tree ensembles probably lose a bit because 353 training rows isn't much to learn from. I didn't tune the nonlinear models, so I can't fully rule out small non-linear effects.

Model	CV R^2	R^2 std	CV RMSE	CV MAE
OLS (baseline)	0.480	0.041	55.395	45.048
Random Forest	0.432	0.068	57.793	47.591
Gradient Boosting	0.423	0.035	58.425	47.582
SVR (RBF)	0.475	0.058	55.591	45.185

Table 6. Linear vs nonlinear models, 5-fold CV on the training data.

Q5. How sensitive are the conclusions to the choice of model?

The conclusion about performance holds up well. In a repeated 5×5 cross-validation, OLS (0.479), Ridge (0.476), Lasso (0.475) and SVR (0.474) are essentially the same, and Random Forest (0.425) and Gradient Boosting (0.410) are lower and more variable. No model gets meaningfully above 0.48, so whichever one I pick, the message is the same: about half of the variation is explained.

Model	CV R^2	R^2 std
OLS	0.479	0.081
Ridge	0.476	0.079
Lasso	0.475	0.077
Random Forest	0.425	0.106
Gradient Boosting	0.410	0.105
SVR (RBF)	0.474	0.083

Table 7. Repeated 5×5 cross-validation across all models (training data).

Conclusions about individual coefficients are more sensitive, but only for the correlated predictors. Across OLS, Ridge and Lasso, s1, s2, s3 and s5 change a lot (s2 goes from 24.6 to 17.1 to 0.0, and s3 even changes sign under Lasso), so anything I say about those features depends on the model. BMI and blood pressure keep similar coefficients in all three. The engineered-feature results also depend on the model, because OLS falls apart there while Lasso holds up.

Q6. Which model gives the best balance of accuracy and interpretability?

I'd go with Ridge regression on the original 10 features. Accuracy is basically tied across OLS, Ridge, Lasso and SVR (CV R^2 about 0.48, RMSE about 55.4 to 55.6), and the tree ensembles are lower. Ridge also had the best CV R^2 of all twelve configurations I compared at the end (0.481), which is why the final evaluation selects it.

Interpretability is what separates the models. Linear models give one coefficient per feature, with a sign and a size I can read directly. Random Forest, Gradient Boosting and SVR need extra tools to explain, and they don't buy any accuracy here. Among the linear models, Ridge matches OLS on accuracy but shrinks the unstable coefficients of the correlated predictors s1, s2 and s3. Lasso is a fair choice if I wanted a sparser model, but with this group of correlated features it picks one somewhat arbitrarily (it zeroes s2 and flips the sign of s3), so its individual coefficients are harder to trust. Plain OLS is fine too if I'm not going to interpret the correlated lipid features one by one.

5. Feature-Engineered Regression

In this part I added powers (x^2, x^3) and pairwise interaction terms ($x_i x_j$) and fit OLS, Ridge and Lasso on the expanded features. I used wider penalty grids here because there are many more correlated features. I looked at three things: what the engineered features do, what the polynomial degree does, and which coefficients each method keeps.

5.1 Effect of the engineered features

Adding squares, cubes and interactions (75 features) did not beat the original 10 features. On the original features the CV R^2 is 0.480 (OLS), 0.481 (Ridge) and 0.477 (Lasso). On the engineered set it drops to 0.293 for OLS and 0.442 for Ridge. Only Lasso stays level, at 0.475.

Features	Model	n	Train R^2	CV R^2	CV std	CV MAE	Gap
Original	OLS	10	0.531	0.480	0.041	45.05	0.050
Original	Ridge	10	0.526	0.481	0.039	45.07	0.045
Original	Lasso	10	0.524	0.477	0.036	45.33	0.047
x^2, x^3, $x_i x_j$	OLS	75	0.647	0.293	0.095	50.66	0.354
x^2, x^3, $x_i x_j$	Ridge	75	0.568	0.442	0.053	46.56	0.126
x^2, x^3, $x_i x_j$	Lasso	75	0.545	0.475	0.044	45.71	0.070

Table 8. Original vs engineered features (5-fold CV on the training data). Gap = train R^2 minus CV R^2.

5.2 Polynomial degree and overfitting

As the degree goes from 1 to 2 to 3, OLS fits the training data better and better (train R^2 0.531, 0.626, 0.980), while its CV R^2 collapses (0.480, 0.339, then −1279). A negative R^2 means the model does worse than just predicting the mean of the target. With 285 features and only about 350 training rows per fold, degree-3 OLS nearly memorises the training data, and its predictions on unseen rows are way off. Ridge degrades gently and Lasso degrades the least, so the more features there are, the more regularization matters.

Degree	Model	n	Train R^2	CV R^2	CV std	CV MAE	Gap
1	OLS	10	0.531	0.480	0.041	45.05	0.050
1	Ridge	10	0.526	0.481	0.039	45.07	0.045
1	Lasso	10	0.524	0.477	0.036	45.33	0.047
2	OLS	65	0.626	0.339	0.070	49.63	0.287
2	Ridge	65	0.559	0.447	0.052	46.81	0.113
2	Lasso	65	0.524	0.460	0.033	46.52	0.064
3	OLS	285	0.980	−1279.029	1488.828	1066.02	1280.0
3	Ridge	285	0.580	0.403	0.030	48.99	0.177
3	Lasso	285	0.527	0.442	0.033	47.29	0.085

Table 9. Effect of polynomial degree (5-fold CV on the training data).

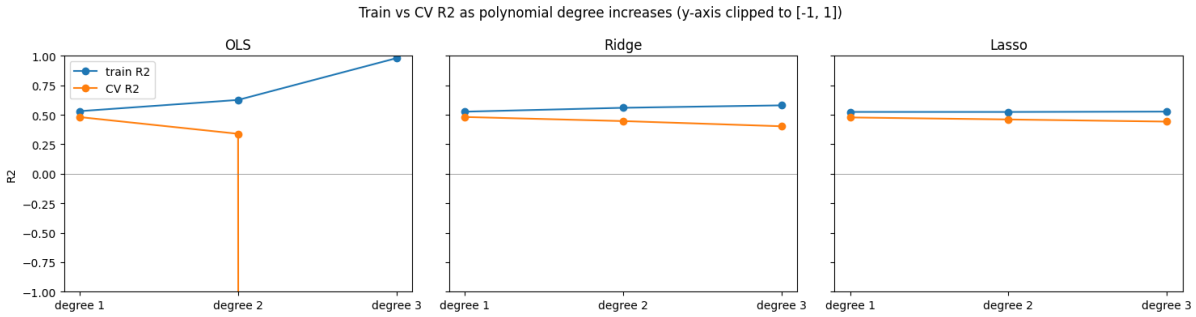


Figure 3. Train vs CV R^2 as the polynomial degree increases (y-axis clipped to [−1, 1]).

5.3 Coefficients selected by each method

On the 75 engineered features, OLS keeps all of them and some get very large (the largest absolute coefficient is 176.2). Its biggest ones are interactions between the highly collinear lipid features, which are unstable. Ridge shrinks everything (largest 15.0) but still keeps all 75. Lasso keeps only 27 of the 75. Under both Ridge and Lasso, the strongest coefficients are the original main effects s5, bmi, bp and s3. Lasso also keeps a few non-linear terms (sex^2, $s5^3$, bmi^3, age×sex), but they are smaller than the main effects. So most of the signal is linear in s5, bmi, bp and s3.

Rank	OLS	Ridge	Lasso
1	s1*s2	s5	s5
2	s1*s5	bmi	bmi
3	s2*s5	bp	s3
4	$s2^2$	s3	bp

Rank	OLS	Ridge	Lasso
5	$s4^2$	bmi^3	sex^2
6	$s5^2$	bp^3	$s5^3$
7	s3*s4	s4	bmi^3
8	$s5^3$	bmi*bp	age*sex
9	s1*s4	$s2^3$	bp^3
10	s3*s5	age*s2	bmi*bp

Table 10. Top 10 features by absolute coefficient for each method (engineered feature set).

6. Final Evaluation on the Held-Out Test Set

I chose and tuned every configuration using the training data only. Table 11 shows each feature set with each model, with the CV R^2 on the training data next to the test-set R^2, RMSE and MAE. The final model is the one with the best CV R^2, so the test set has no say in which model gets picked.

Features	Model	CV R^2 (train)	Test R^2	Test RMSE	Test MAE
Original	OLS	0.480	0.453	53.853	42.794
Original	**Ridge**	**0.481**	**0.454**	**53.773**	**42.813**
Original	Lasso	0.477	0.471	52.923	42.787
x^2, x^3, x_ix_j	OLS	0.293	0.411	55.840	44.901
x^2, x^3, x_ix_j	Ridge	0.442	0.472	52.886	43.283
x^2, x^3, x_ix_j	Lasso	0.475	0.518	50.543	40.635
Degree 2	OLS	0.339	0.416	55.642	43.582
Degree 2	Ridge	0.447	0.491	51.912	42.111
Degree 2	Lasso	0.460	0.516	50.626	41.196
Degree 3	OLS	−1279.029	−14.561	287.134	164.854
Degree 3	Ridge	0.403	0.414	55.713	46.011
Degree 3	Lasso	0.442	0.479	52.520	43.532

Table 11. CV R^2 on the training data and test-set metrics for all twelve configurations. The selected model (original features, Ridge) is in bold.

Final model. Selecting by CV R^2 picks Ridge on the original features (CV R^2 0.481). On the test set it reaches an R^2 of 0.454, an RMSE of 53.8 and an MAE of 42.8. That is close to the CV estimate, so cross-validation gave me a fair picture of how the model generalizes.

A note on the other test scores. Lasso on the engineered features (test R^2 0.518) and on degree 2 (0.516) scored higher on the test set than the model I chose. Their CV R^2 wasn't higher (0.475 and 0.460), and the test set has only 89 rows, so a gap of about 0.06 could easily be noise. Switching to them because of the test result would mean using the test set to select a model, so I stuck with the CV choice. Degree-3 OLS has a test R^2 of −14.6, which confirms the overfitting I saw in cross-validation.

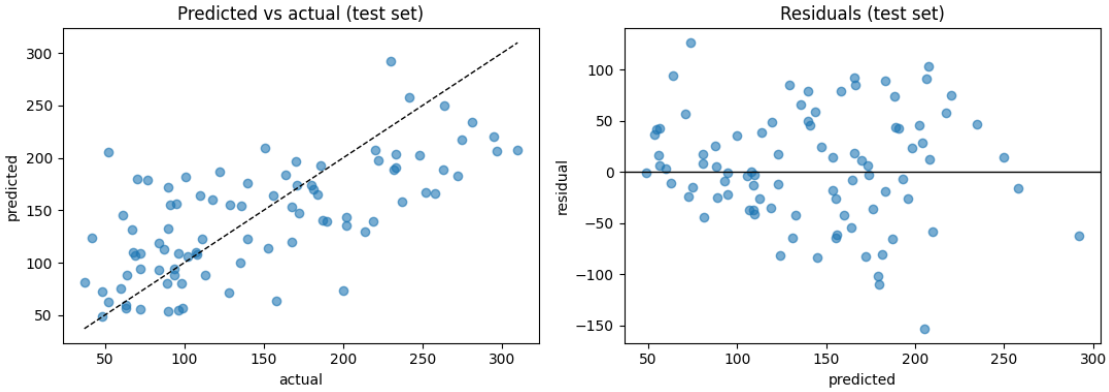


Figure 4. Predicted vs actual values and residuals on the test set for the selected model.

7. Conclusion

Coming back to the central question of how feature engineering, model complexity and regularization affect OLS, Ridge and Lasso, here is what I found.

- **Feature engineering didn't improve prediction.** The best CV R^2 stays at about 0.48, and the original features already reach it.
- **More complexity hurts OLS badly.** Its CV R^2 falls from 0.480 to 0.339 to −1279 as the degree goes from 1 to 3. Ridge degrades gently, and Lasso is the most robust.
- **Regularization matters more as features multiply.** With 10 features it doesn't improve accuracy, but it does steady the coefficients of the correlated lipid features. Lasso also selects features, keeping 27 of the 75 on the engineered set.
- **The signal is mostly linear.** It is carried mainly by s5, bmi, bp and s3. About half of the variation in progression is still unexplained by every model I tried.

Limitations

- The dataset is small (442 rows) and the test set has only 89 rows, so a single test score is noisy.
- I didn't tune the nonlinear models, so I can't fully rule out small non-linear effects.
- The correlations and coefficients show association, not causation.
- The features come already scaled by scikit-learn and I don't have the original measurements, so I can't interpret the coefficients in clinical units.